UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Faculty of Engineering, Built Environment and Information Technology

EBB 320

Control Systems

Practical 1: Hardware Implementation and System Identification

Name and Surname	Student Number	Signature	% Contribution
S. Tudent	12345678		20
Cornelis Claasen	24704611		33
S.T.U. Dent	34567890		45

By signing this assignment we confirm that we have read and are aware of the University of Pretoria's policy on academic dishonesty and plagiarism and we declare that the work submitted in this assignment is our own as delimited by the mentioned policies. We explicitly declare that no parts of this assignment have been copied from current or previous students' work or any other sources (including the internet), whether copyrighted or not. We understand that we will be subjected to disciplinary actions should it be found that the work we submit here does not comply with the said policies.

August 13, 2026

Contents

1 Theoretical Background

2 Software and Hardware Design Implementation

2.1 MCU Code

In order to allow for easier future interaction, a command system was introduced. The MCU receives commands over UART as a string with the format:

`"<command>, <value>"`

These commands are interpreted by the MCU, changing the appropriate values.

Command	Value Changed
0	Change P
1	Change I
2	Change D
3	Change Setpoint
4	Change Disturbance Value
5	Change Duty Cycle
6	Change PWM Output
7	Change Start Flag

If the start variable is set, then the sampling process begins. First a timer is started, the PWM output flags variable is then checked to see which PWM signals must be set, and finally the samples are taken.

The samples are converted to temperatures using the temperature calibration equation. A running average of 200 samples is used to filer high frequency noise. The timer started at the beginning of the sampling process is then checked to see if the sampling period has passed, and if not, it waits for the remaining time. The processed samples are again sent over UART as a string with the format:

`"<Temp1>, <Temp2>, <PWM1>, <PWM2>, <Time>, <Message>"`

The `<Message>` paramater is used to signal error data. Currently the only error message implemented is for if the sampling process takes longer than the set sampling period. (Currently 200 Hz)

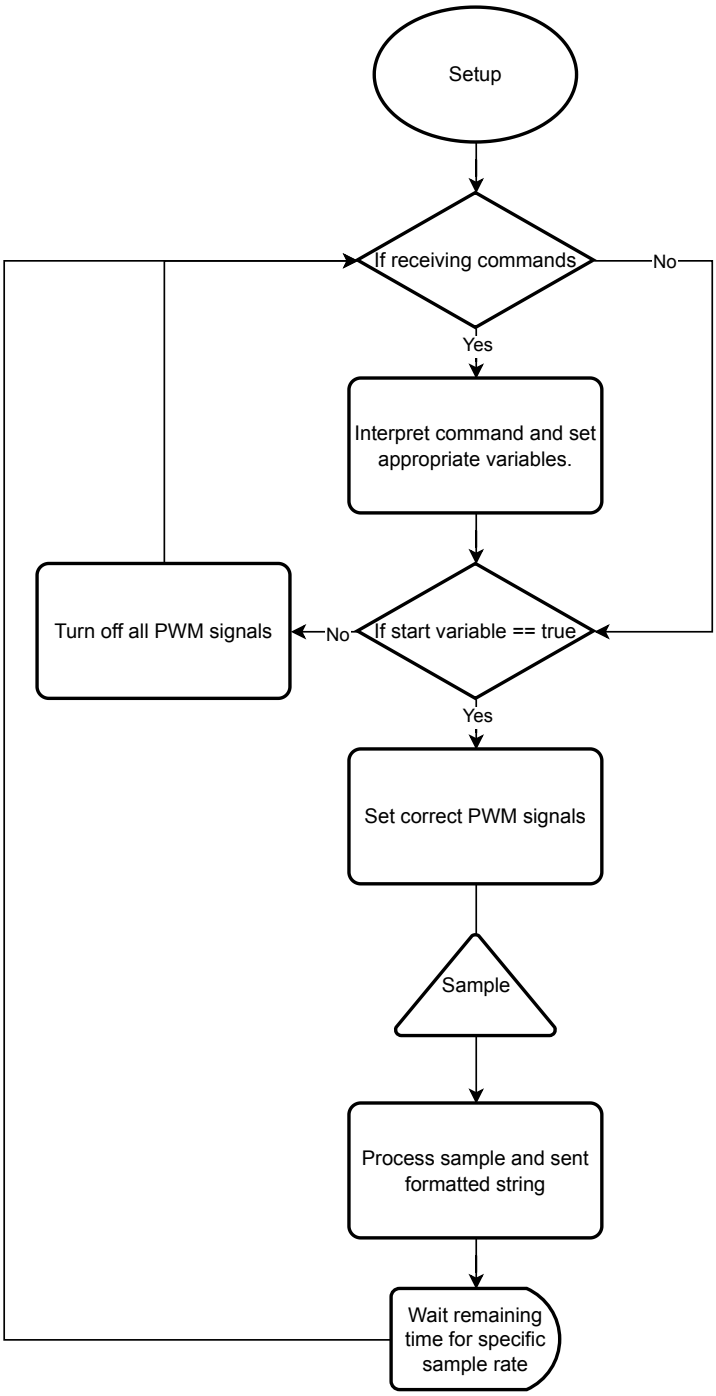

Figure 1: Flow Diagram of MCU Code.

2.2 Matlab Code

The Matlab code was written into a Matlab app to make it easier to operate the system. The app waits for the start button to be pressed, after which it sends the commands to the MCU to update all of the parameters set.

It then sets up the plotting variables and a variable called "capturing", which is used to continue the plotting process in a while loop, until the stop button is pressed. The loop continually checks if it has received data, and if it has, it interprets the formatted string send by the MCU. It plots the received data, checks the error messages received, and displays information based on the received message.

Before the loop starts a .csv file is accessed (or created if it did not exist beforehand), and the formatted string received from the MCU is saved directly to it.

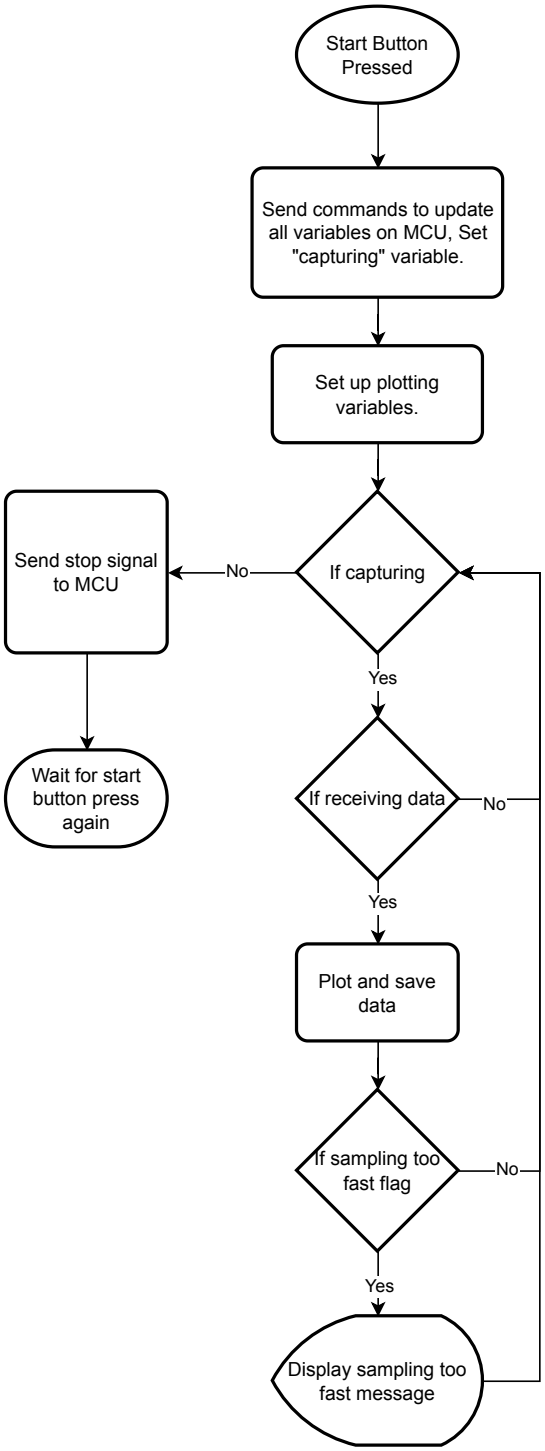

Figure 2: Flow Diagram of Matlab Code.

2.3 Hardware Development

2.3.1 Circuit Design

The physical system consists of two identical channels, each built around a TIP31C power transistor and an MCP9700 linear temperature sensor. In this application the transistor does not operate as a signal amplifier; instead, it functions as a PWM-controlled current driver. The base of each transistor is switched by a PWM signal generated on the ESP32, which sets the average current delivered from the 8 V supply through the transistor into the heat sink. This produces a direct causal chain from the controllable input to the measured output:

$$\text{PWM duty cycle} \rightarrow \text{transistor conduction current} \rightarrow \text{power dissipated as heat} \rightarrow \text{heat sink temperature} \rightarrow \text{sensor output voltage}$$

This chain constitutes the physical plant that is later identified as a transfer function in the third section of this report.

Because the ESP32 operates its GPIO logic at 3.3 V, the base resistor values were selected according to the 3.3 V column of Table 1 in the practical guide: $R_1 = 270\ \Omega$ and $R_2 = 560\ \Omega$. These resistors were chosen to limit the base current drawn from the ESP32 GPIO pin to a safe level while still supplying enough current to fully switch the TIP31C on, given the fixed 8 V bench-supply rail feeding the collector.

The MCP9700 temperature sensor was selected as it requires no external signal-conditioning circuitry; its output pin can be connected directly to an ESP32 ADC input. The sensor was mounted in direct physical contact with the TIP31C package (and, by extension, the heat sink) to ensure accurate thermal coupling between the heat source and the measurement point.

2.3.2 Implementation

The circuit was constructed on a breadboard following the layout described in the practical guide, with both transistor/sensor pairs mounted such that their heat sinks were in direct contact with one another, ensuring consistent thermal cross-coupling between channels for later system identification of the off-diagonal transfer functions (G_{12}, G_{21}). The 8 V supply rail was provided by a bench power supply set to its maximum allowable current limit, as an ESP32 GPIO pin cannot source sufficient current to drive the transistor directly. The ESP32 shared a common ground with the transistor circuit's power supply to ensure a consistent reference voltage across the system.

2.3.3 Temperature Calibration Equation

The MCP9700 does not output a digital temperature value; it is a purely analogue device whose output voltage varies linearly with the temperature. Microchip characterises this relationship in the datasheet as a first-order (linear) transfer function:

$$V_{OUT} = T_C \cdot T_A + V_{0^\circ C} \tag{1}$$

where V_{OUT} is the sensor's output voltage, T_A is the ambient temperature, T_C is the temperature coefficient (the slope of the line, in V/°C), and $V_{0^\circ C}$ is the sensor's

output voltage at 0°C (the y-intercept). This is a standard linear equation of the form $y = mx + c$.

For the MCP9700/9700A/9700B family, the datasheet's DC Electrical Characteristics table specifies:

$$T_C = 10.0 \text{ mV/°C}, \qquad V_{0°C} = 500 \text{ mV} \tag{2}$$

Since the ADC measures a voltage and the objective is to recover a temperature, Equation (1) is rearranged to make T_A the subject:

$$T_A = \frac{V_{OUT} - V_{0°C}}{T_C} \tag{3}$$

Substituting the datasheet constants, with the voltage values expressed in volts ($V_{0°C} = 0.5\,\text{V}$, $T_C = 0.01\,\text{V/°C}$):

$$T_A(\text{°C}) = \frac{V_{ADC} - 0.5}{0.01} \tag{4}$$

Equation (4) is identical to Equation 1 given in the practical guide. The substitution of V_{ADC} for V_{OUT} reflects the physical circuit: since the sensor output is connected directly to the ESP32's ADC input with no intermediate signal conditioning, the voltage sampled by the ADC is equal to the sensor's output voltage.

The sensor required no additional physical calibration step; it is a factory-characterised property of the device. The datasheet specifies a typical accuracy of ±4°C over the 0–70°C range for the MCP9700, which represents an inherent source of measurement uncertainty carried into the system identification results discussed later in this report.

3 Results

3.1 Derived FOPDT transfer functions

The transfer functions of the system were derived using two different methods, namely the process reaction curve method and transfer function fitting using MATLAB. Each method yielded a MIMO transfer function matrix with the following mathematical description:

$$\begin{bmatrix} Y_1(s) \\ Y_2(s) \end{bmatrix} = \begin{bmatrix} G_{11}(s) & G_{12}(s) \\ G_{21}(s) & G_{22}(s) \end{bmatrix} \begin{bmatrix} U_1(s) \\ U_2(s) \end{bmatrix} \tag{5}$$

The following table contains these two matrices.

Table 1: Comparison of Transfer Functions: Transfer Function Fitting vs. Process Reaction Curve

Thermal Pathway	**Transfer Function Fitting (`tfest`)**	**Process Reaction Curve**
$G_{11}(s)$ (PWM 1 $\rightarrow$ T1)	$\frac{-0.1974s+0.05300}{s^2+0.4582s+0.0034}$	$\frac{K_{11}}{\tau_{11}s+1}e^{-\theta_{11}s}$
$G_{12}(s)$ (PWM 2 $\rightarrow$ T1)	$\frac{-0.0156s+0.0010}{s^2+0.0300s+0.0002}$	$\frac{K_{12}}{\tau_{12}s+1}e^{-\theta_{12}s}$
$G_{21}(s)$ (PWM 1 $\rightarrow$ T2)	$\frac{-0.0097s+0.0016}{s^2+0.0268s+0.0002}$	$\frac{K_{21}}{\tau_{21}s+1}e^{-\theta_{21}s}$
$G_{22}(s)$ (PWM 2 $\rightarrow$ T2)	$\frac{-0.0346s+0.0234}{s^2+0.3998s+0.0024}$	$\frac{K_{22}}{\tau_{22}s+1}e^{-\theta_{22}s}$

Comment on the table

3.2 Matlab fitting

The **tfest** function was used in Matlab to fit the data to a transfer function. The following steps were followed to achieve this:

Data Preparation:
The recorded data was sliced to include a sufficient amount of zero inputs, this is required by the tfest function to properly fit the model. After which the data was broken into a voltage vector (the PWM input), and two temperature vectors (temperatures of each of the transistors). The ambient temperature was subtracted from the temperature vectors and the data was packaged into an iddata object.

Model Fitting:
Three parameters were passed to the tfest function, namely: the iddata object, the number of poles the system has, and the number of zeros the system has. The number of poles a system has is determined by the number of energy storing elements, in the case of this system the transistors are storing energy in the form of heat, thus

the system has two poles. It is common practise to take the number of zeros of an unknown system as: Zeros = Poles -1. The fitted models were then plotted using the compare function as can be seen in the Figure 3 and Figure **??** below.

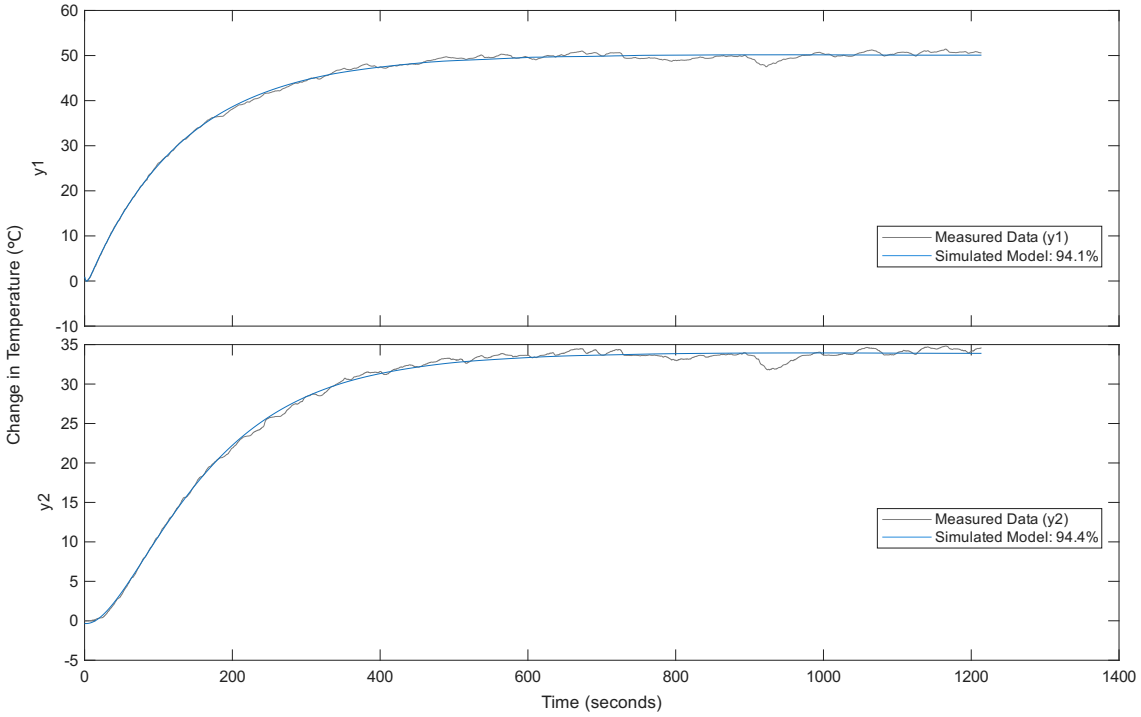

Figure 3: Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 1.

Figure 3 displays the fitted model for both G_{11} (top) and G_{21} (bottom) which are similarly accurate; 94.1% and 94.4%, on top of the measured physical data.

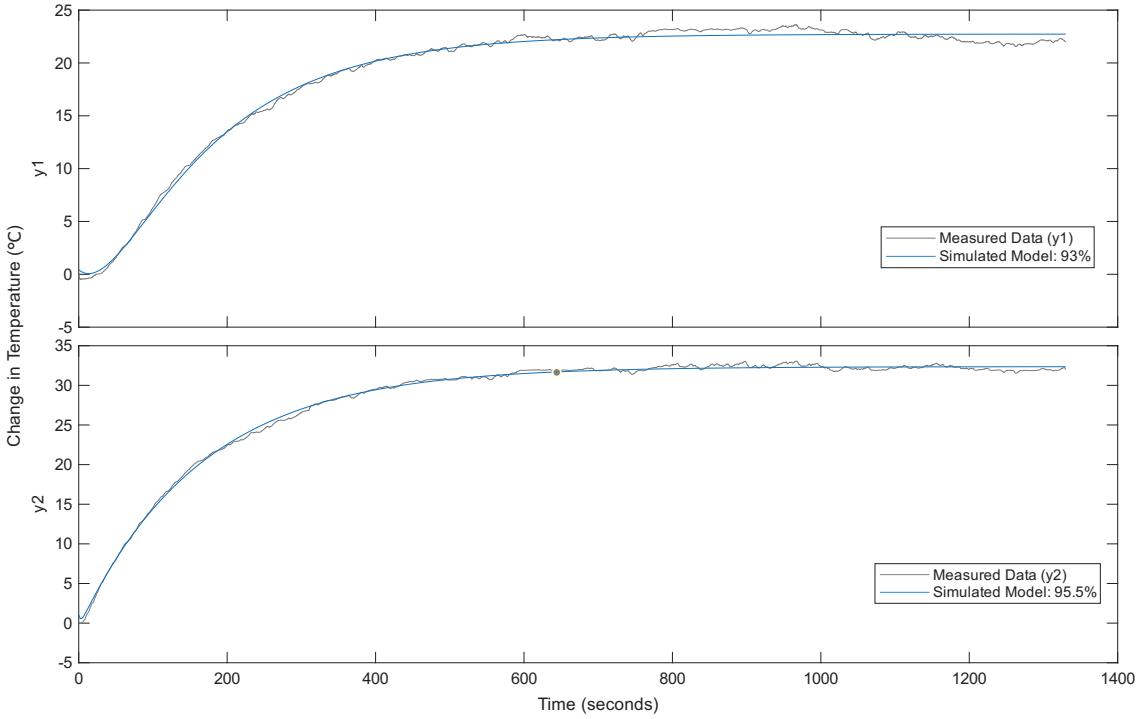

Figure 4: Comparison of the physical sensor data against the simulated SIMO transfer function model for PWM 2.

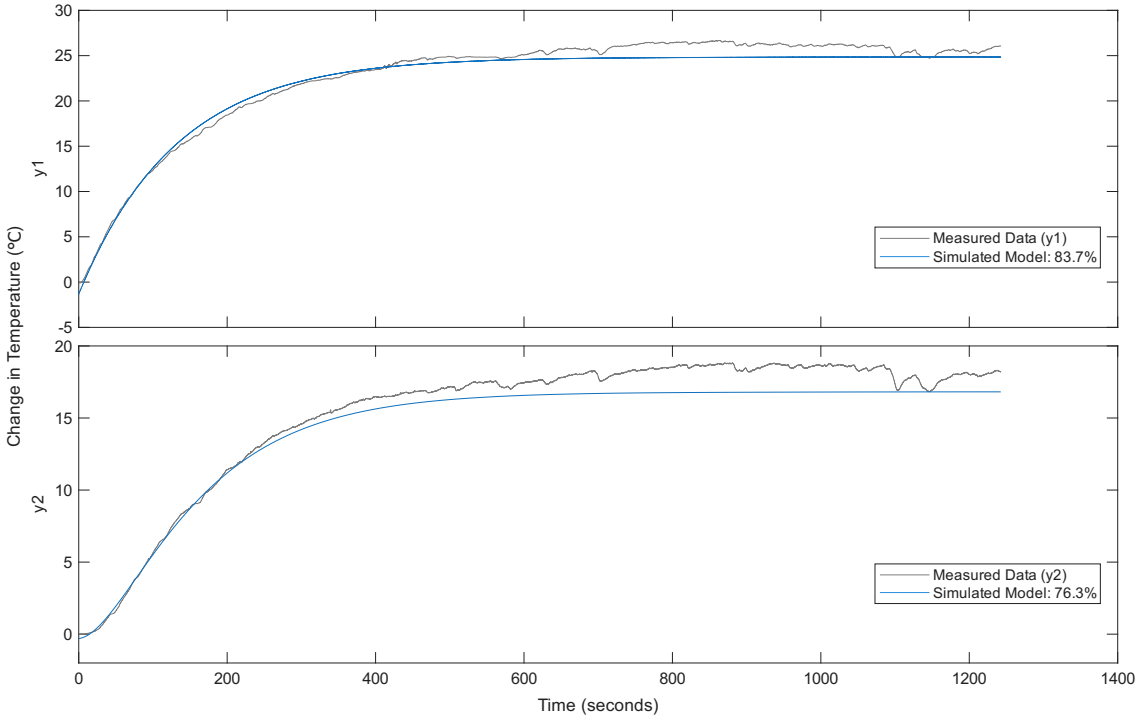

Figure 5: Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 1.

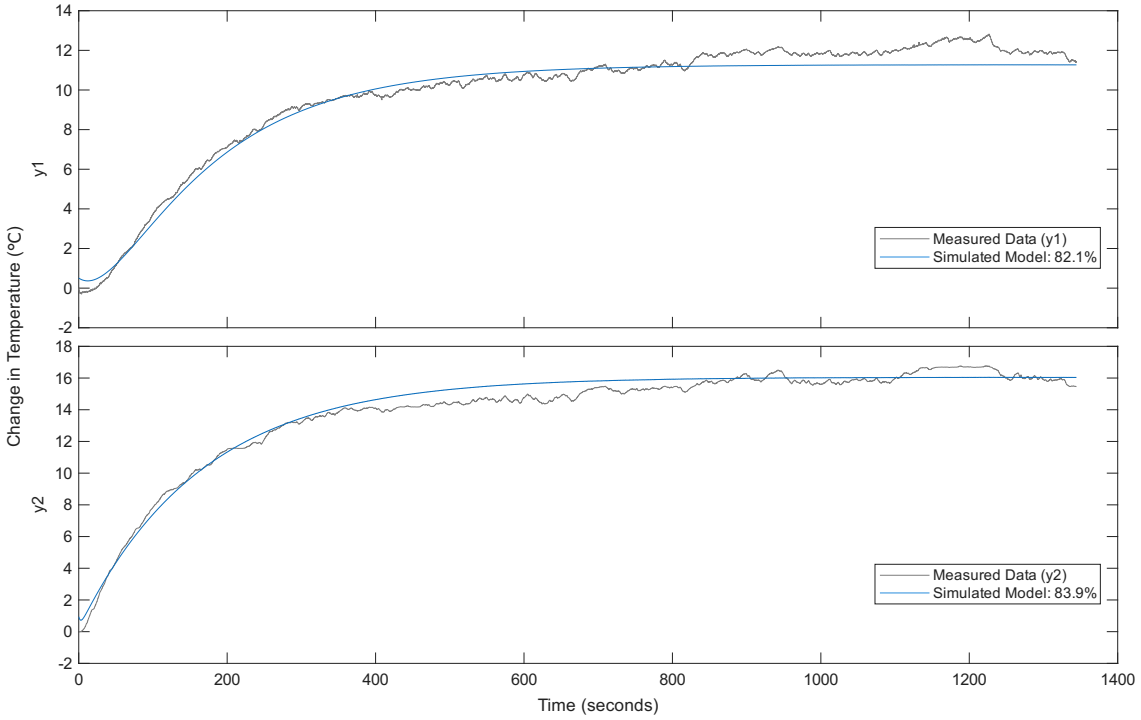

Figure 6: Comparison of the physical sensor data for a 50% duty cycle test case against the simulated SIMO transfer function model for PWM 2.

4 Discussion

4.1 Model validation

The built-in **compare** function in MATLAB was used to determine the model accuracy. This function utilises the normalised root mean square error (NRMSE) fitness metric.

5 Code

5.1 Matlab Code

Listing 1: App Code For Sampling

```
classdef EBBPrac < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                    matlab.ui.Figure
        GridLayout                  matlab.ui.container.
           GridLayout
        LeftPanel                   matlab.ui.container.
           Panel
        TabGroup                    matlab.ui.container.
           TabGroup
        Tab                         matlab.ui.container.
           Tab
        DataSaveEditField           matlab.ui.control.
           EditField
        DataSaveEditFieldLabel      matlab.ui.control.
           Label
        SamplingtoofastLabel        matlab.ui.control.
           Label
        UpdateChannelsButton        matlab.ui.control.
           Button
        StartButton                 matlab.ui.control.
           StateButton
        StopButton                  matlab.ui.control.
           StateButton
        OutputDropDown              matlab.ui.control.
           DropDown
        OutputDropDownLabel         matlab.ui.control.
           Label
        DutyCycleEditField          matlab.ui.control.
           NumericEditField
        DutyCycleEditFieldLabel     matlab.ui.control.
           Label
        DisturbanceEditField        matlab.ui.control.
           NumericEditField
        DisturbanceEditFieldLabel   matlab.ui.control.
           Label
        SetpointEditField           matlab.ui.control.
           NumericEditField
        SetpointEditFieldLabel      matlab.ui.control.
           Label
        DEditField                  matlab.ui.control.
           NumericEditField
```

```
        DEditFieldLabel             matlab.ui.control.
           Label
        IEditField                  matlab.ui.control.
           NumericEditField
        IEditFieldLabel             matlab.ui.control.
           Label
        PEditField                  matlab.ui.control.
           NumericEditField
        PEditFieldLabel             matlab.ui.control.
           Label
        Tab2                        matlab.ui.container.
           Tab
        BaudRateEditField           matlab.ui.control.
           NumericEditField
        BaudRateEditFieldLabel      matlab.ui.control.
           Label
        COMPortEditField            matlab.ui.control.
           EditField
        COMPortEditFieldLabel       matlab.ui.control.
           Label
        RightPanel                  matlab.ui.container.
           Panel
        UIAxes_2                    matlab.ui.control.
           UIAxes
        UIAxes                      matlab.ui.control.
           UIAxes
    end

    % Properties that correspond to apps with auto-reflow
    properties (Access = private)
        onePanelWidth = 576;
    end


    properties (Access = private)
        %Live lines 1 and 2 are for the temperature
           measurements.
        liveLine1
        liveLine2
        %Live lines 3 and 4 are for the PWM signals.
        liveLine3
        liveLine4
        P
        I
        D
        setpoint
        disturbance
        duty_cycle
```

```
        output
        capturing
        COM_port
        baud_rate
        window_size
        esp32
        data_save_name % Description
    end


    % Callbacks that handle component events
    methods (Access = private)

        % Value changed function: PEditField
        function PEditFieldValueChanged(app, event)
            app.P = app.PEditField.Value;
        end

        % Value changed function: IEditField
        function IEditFieldValueChanged(app, event)
            app.I = app.IEditField.Value;
        end

        % Value changed function: DEditField
        function DEditFieldValueChanged(app, event)
            app.D = app.DEditField.Value;
        end

        % Value changed function: SetpointEditField
        function SetpointEditFieldValueChanged(app, event
           )
            app.setpoint = app.SetpointEditField.Value;
        end

        % Value changed function: DisturbanceEditField
        function DisturbanceEditFieldValueChanged(app,
           event)
            app.disturbance = app.DisturbanceEditField.
               Value;
        end

        % Value changed function: DutyCycleEditField
        function DutyCycleEditFieldValueChanged(app,
           event)
            app.duty_cycle = app.DutyCycleEditField.Value
               ;
        end
```

```
% Value changed function: OutputDropDown
function OutputDropDownValueChanged(app, event)
    app.output = str2double(app.OutputDropDown.
       Value);
end

% Value changed function: StartButton
function StartButtonValueChanged(app, event)
    app.StartButton.Visible = "off";
    app.StopButton.Visible = "on";
    app.UpdateChannelsButton.Visible = "on";
    app.capturing = true;

    %Call all of the other functions to make sure
        their values are
    %captured.
    app.PEditFieldValueChanged();
    app.IEditFieldValueChanged();
    app.DEditFieldValueChanged();
    app.SetpointEditFieldValueChanged();
    app.DisturbanceEditFieldValueChanged();
    app.DutyCycleEditFieldValueChanged();
    app.OutputDropDownValueChanged();
    app.BaudRateEditFieldValueChanged();
    app.COMPortEditFieldValueChanged();
    app.DataSaveEditFieldValueChanged();

    if isempty(app.esp32)
        app.esp32 = serialport(app.COM_port, app.
           baud_rate);
        configureTerminator(app.esp32, "LF");
    end
    flush(app.esp32);

    pause(1);

    %Make sure all info is read into esp.
    writeline(app.esp32, sprintf("%d, %.2f", 0,
       app.P)); %
    writeline(app.esp32, sprintf("%d, %.2f", 1,
       app.I)); %
    writeline(app.esp32, sprintf("%d, %.2f", 2,
       app.D)); %
    writeline(app.esp32, sprintf("%d, %.2f", 3,
       app.setpoint)); %
    writeline(app.esp32, sprintf("%d, %.2f", 4,
       app.disturbance)); %
```

```
writeline(app.esp32, sprintf("%d, %d", 5, app
    .duty_cycle)); %
writeline(app.esp32, sprintf("%d, %.2f", 6,
    app.output)); %
writeline(app.esp32, sprintf("%d, %.2f", 7,
    app.capturing)); %

%Clear axes.
cla(app.UIAxes);
cla(app.UIAxes_2);

%Plotting axes for the temperature
    measurements.
app.liveLine1 = animatedline(app.UIAxes, '
    Color', 'r');
app.liveLine2 = animatedline(app.UIAxes, '
    Color', 'b');

%Plotting axes for the PWM signals.
app.liveLine3 = animatedline(app.UIAxes_2, '
    Color', 'r');
app.liveLine4 = animatedline(app.UIAxes_2, '
    Color', 'b');

app.window_size = 30; %seconds

%Define data csv that we will safe to.
fid = fopen(app.data_save_name + ".csv", 'wt'
    );

while(app.capturing)
    if app.esp32.NumBytesAvailable > 0
        rawData = readline(app.esp32);
        data = str2double(split(rawData, ',')
            );
        data(5) = data(5)/1000;

        if ~isnan(data(1)) && (length(data)
            == 6)
            %Plot temperature data.
            addpoints(app.liveLine1, data(5),
                data(1));
            addpoints(app.liveLine2, data(5),
                data(2));
            xlim(app.UIAxes, [data(5) - app.
                window_size, data(5)]);

            %Plot PWM signals.
```

```
                addpoints(app.liveLine3, data(5),
                    data(3));
                addpoints(app.liveLine4, data(5),
                    data(4));
                xlim(app.UIAxes_2, [data(5) - app
                   .window_size, data(5)]);

                %Write data to file.
                fprintf(fid, rawData);

                if data(end) == 0
                    app.SamplingtoofastLabel.
                       Visible = "on";
                else
                    app.SamplingtoofastLabel.
                       Visible = "off";
                end
            end

            drawnow limitrate
        end
    end

    %Close file we are saving to.
    fclose(fid);

    %Send stop signal.
    writeline(app.esp32, sprintf("%d, %.2f", 7,
       app.capturing));

    clear app.esp32;
end

% Value changed function: COMPortEditField
function COMPortEditFieldValueChanged(app, event)
    app.COM_port = app.COMPortEditField.Value;
end

% Value changed function: BaudRateEditField
function BaudRateEditFieldValueChanged(app, event
   )
    app.baud_rate = app.BaudRateEditField.Value;
end

% Value changed function: StopButton
function StopButtonValueChanged(app, event)
    app.capturing = false;
    app.StopButton.Visible = "off";
```

```
            app.UpdateChannelsButton.Visible = "off";
            app.StartButton.Visible = "on";

        end

        % Button pushed function: UpdateChannelsButton
        function UpdateChannelsButtonPushed(app, event)
            app.DutyCycleEditFieldValueChanged();
            app.OutputDropDownValueChanged();
            writeline(app.esp32, sprintf("%d, %.2f", 5,
               app.duty_cycle));
            writeline(app.esp32, sprintf("%d, %.2f", 6,
               app.output));
        end

        % Value changed function: DataSaveEditField
        function DataSaveEditFieldValueChanged(app, event
           )
            app.data_save_name = app.DataSaveEditField.
               Value;

        end

        % Changes arrangement of the app based on
           UIFigure width
        function updateAppLayout(app, event)
            currentFigureWidth = app.UIFigure.Position(3)
               ;
            if(currentFigureWidth <= app.onePanelWidth)
                % Change to a 2x1 grid
                app.GridLayout.RowHeight = {449, 449};
                app.GridLayout.ColumnWidth = {'1x'};
                app.RightPanel.Layout.Row = 2;
                app.RightPanel.Layout.Column = 1;
            else
                % Change to a 1x2 grid
                app.GridLayout.RowHeight = {'1x'};
                app.GridLayout.ColumnWidth = {204, '1x'};
                app.RightPanel.Layout.Row = 1;
                app.RightPanel.Layout.Column = 2;
            end
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
```

```
function createComponents(app)

    % Create UIFigure and hide until all
       components are created
    app.UIFigure = uifigure('Visible', 'off');
    app.UIFigure.AutoResizeChildren = 'off';
    app.UIFigure.Position = [100 100 702 449];
    app.UIFigure.Name = 'MATLAB App';
    app.UIFigure.SizeChangedFcn =
       createCallbackFcn(app, @updateAppLayout,
       true);

    % Create GridLayout
    app.GridLayout = uigridlayout(app.UIFigure);
    app.GridLayout.ColumnWidth = {204, '1x'};
    app.GridLayout.RowHeight = {'1x'};
    app.GridLayout.ColumnSpacing = 0;
    app.GridLayout.RowSpacing = 0;
    app.GridLayout.Padding = [0 0 0 0];
    app.GridLayout.Scrollable = 'on';

    % Create LeftPanel
    app.LeftPanel = uipanel(app.GridLayout);
    app.LeftPanel.Layout.Row = 1;
    app.LeftPanel.Layout.Column = 1;

    % Create TabGroup
    app.TabGroup = uitabgroup(app.LeftPanel);
    app.TabGroup.Position = [3 3 197 442];

    % Create Tab
    app.Tab = uitab(app.TabGroup);
    app.Tab.Title = 'Tab';

    % Create PEditFieldLabel
    app.PEditFieldLabel = uilabel(app.Tab);
    app.PEditFieldLabel.HorizontalAlignment = '
       right';
    app.PEditFieldLabel.Position = [46 385 25
       22];
    app.PEditFieldLabel.Text = 'P';

    % Create PEditField
    app.PEditField = uieditfield(app.Tab, '
       numeric');
    app.PEditField.ValueChangedFcn =
       createCallbackFcn(app, @
       PEditFieldValueChanged, true);
```

```
app.PEditField.Position = [85 385 100 22];

% Create IEditFieldLabel
app.IEditFieldLabel = uilabel(app.Tab);
app.IEditFieldLabel.HorizontalAlignment = '
   right';
app.IEditFieldLabel.Position = [46 355 25
   22];
app.IEditFieldLabel.Text = 'I';

% Create IEditField
app.IEditField = uieditfield(app.Tab, '
   numeric');
app.IEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   IEditFieldValueChanged, true);
app.IEditField.Position = [85 355 100 22];

% Create DEditFieldLabel
app.DEditFieldLabel = uilabel(app.Tab);
app.DEditFieldLabel.HorizontalAlignment = '
   right';
app.DEditFieldLabel.Position = [46 326 25
   22];
app.DEditFieldLabel.Text = 'D';

% Create DEditField
app.DEditField = uieditfield(app.Tab, '
   numeric');
app.DEditField.ValueChangedFcn =
   createCallbackFcn(app, @
   DEditFieldValueChanged, true);
app.DEditField.Position = [85 326 100 22];

% Create SetpointEditFieldLabel
app.SetpointEditFieldLabel = uilabel(app.Tab)
   ;
app.SetpointEditFieldLabel.
   HorizontalAlignment = 'right';
app.SetpointEditFieldLabel.Position = [21 296
    49 22];
app.SetpointEditFieldLabel.Text = 'Setpoint';

% Create SetpointEditField
app.SetpointEditField = uieditfield(app.Tab,
   'numeric');
app.SetpointEditField.ValueChangedFcn =
   createCallbackFcn(app, @
```

```
    SetpointEditFieldValueChanged, true);
app.SetpointEditField.Position = [85 296 100
    22];

% Create DisturbanceEditFieldLabel
app.DisturbanceEditFieldLabel = uilabel(app.
    Tab);
app.DisturbanceEditFieldLabel.
    HorizontalAlignment = 'right';
app.DisturbanceEditFieldLabel.Position = [1
    265 69 22];
app.DisturbanceEditFieldLabel.Text = '
    Disturbance';

% Create DisturbanceEditField
app.DisturbanceEditField = uieditfield(app.
    Tab, 'numeric');
app.DisturbanceEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DisturbanceEditFieldValueChanged, true);
app.DisturbanceEditField.Position = [85 265
    100 22];

% Create DutyCycleEditFieldLabel
app.DutyCycleEditFieldLabel = uilabel(app.Tab
    );
app.DutyCycleEditFieldLabel.
    HorizontalAlignment = 'right';
app.DutyCycleEditFieldLabel.Position = [9 235
     63 22];
app.DutyCycleEditFieldLabel.Text = 'Duty
    Cycle';

% Create DutyCycleEditField
app.DutyCycleEditField = uieditfield(app.Tab,
     'numeric');
app.DutyCycleEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DutyCycleEditFieldValueChanged, true);
app.DutyCycleEditField.Position = [86 235 100
     22];
app.DutyCycleEditField.Value = 100;

% Create OutputDropDownLabel
app.OutputDropDownLabel = uilabel(app.Tab);
app.OutputDropDownLabel.HorizontalAlignment =
     'right';
```

```
app.OutputDropDownLabel.Position = [30 199 41
    22];
app.OutputDropDownLabel.Text = 'Output';

% Create OutputDropDown
app.OutputDropDown = uidropdown(app.Tab);
app.OutputDropDown.Items = {'None', 'Channel1
   ', 'Channel2', 'Both'};
app.OutputDropDown.ItemsData = {'3', '0', '1'
   , '2'};
app.OutputDropDown.ValueChangedFcn =
   createCallbackFcn(app, @
   OutputDropDownValueChanged, true);
app.OutputDropDown.Position = [85 199 100
   22];
app.OutputDropDown.Value = '3';

% Create StopButton
app.StopButton = uibutton(app.Tab, 'state');
app.StopButton.ValueChangedFcn =
   createCallbackFcn(app, @
   StopButtonValueChanged, true);
app.StopButton.Visible = 'off';
app.StopButton.Text = 'Stop';
app.StopButton.Position = [85 168 100 23];

% Create StartButton
app.StartButton = uibutton(app.Tab, 'state');
app.StartButton.ValueChangedFcn =
   createCallbackFcn(app, @
   StartButtonValueChanged, true);
app.StartButton.Text = 'Start';
app.StartButton.Position = [86 168 100 23];

% Create UpdateChannelsButton
app.UpdateChannelsButton = uibutton(app.Tab,
   'push');
app.UpdateChannelsButton.ButtonPushedFcn =
   createCallbackFcn(app, @
   UpdateChannelsButtonPushed, true);
app.UpdateChannelsButton.Visible = 'off';
app.UpdateChannelsButton.Position = [81 136
   108 23];
app.UpdateChannelsButton.Text = 'Update
   Channels';

% Create SamplingtoofastLabel
app.SamplingtoofastLabel = uilabel(app.Tab);
```

```
app.SamplingtoofastLabel.Visible = 'off';
app.SamplingtoofastLabel.Position = [90 12
    101 22];
app.SamplingtoofastLabel.Text = 'Sampling too
    fast!';

% Create DataSaveEditFieldLabel
app.DataSaveEditFieldLabel = uilabel(app.Tab)
    ;
app.DataSaveEditFieldLabel.
    HorizontalAlignment = 'right';
app.DataSaveEditFieldLabel.Position = [10 96
    61 22];
app.DataSaveEditFieldLabel.Text = 'Data Save'
    ;

% Create DataSaveEditField
app.DataSaveEditField = uieditfield(app.Tab,
    'text');
app.DataSaveEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    DataSaveEditFieldValueChanged, true);
app.DataSaveEditField.Position = [86 96 100
    22];
app.DataSaveEditField.Value = 'PWM1StepV0';

% Create Tab2
app.Tab2 = uitab(app.TabGroup);
app.Tab2.Title = 'Tab2';

% Create COMPortEditFieldLabel
app.COMPortEditFieldLabel = uilabel(app.Tab2)
    ;
app.COMPortEditFieldLabel.HorizontalAlignment
    = 'right';
app.COMPortEditFieldLabel.Position = [13 385
    58 22];
app.COMPortEditFieldLabel.Text = 'COM Port';

% Create COMPortEditField
app.COMPortEditField = uieditfield(app.Tab2,
    'text');
app.COMPortEditField.ValueChangedFcn =
    createCallbackFcn(app, @
    COMPortEditFieldValueChanged, true);
app.COMPortEditField.Position = [85 385 100
    22];
app.COMPortEditField.Value = '/dev/ttyUSB0';
```

```
            % Create BaudRateEditFieldLabel
            app.BaudRateEditFieldLabel = uilabel(app.Tab2
               );
            app.BaudRateEditFieldLabel.
               HorizontalAlignment = 'right';
            app.BaudRateEditFieldLabel.Position = [9 355
               62 22];
            app.BaudRateEditFieldLabel.Text = 'Baud Rate'
               ;

            % Create BaudRateEditField
            app.BaudRateEditField = uieditfield(app.Tab2,
                'numeric');
            app.BaudRateEditField.ValueChangedFcn =
               createCallbackFcn(app, @
               BaudRateEditFieldValueChanged, true);
            app.BaudRateEditField.Position = [85 355 100
               22];
            app.BaudRateEditField.Value = 115200;

            % Create RightPanel
            app.RightPanel = uipanel(app.GridLayout);
            app.RightPanel.Layout.Row = 1;
            app.RightPanel.Layout.Column = 2;

            % Create UIAxes
            app.UIAxes = uiaxes(app.RightPanel);
            title(app.UIAxes, 'Title')
            xlabel(app.UIAxes, 'X')
            ylabel(app.UIAxes, 'Y')
            zlabel(app.UIAxes, 'Z')
            app.UIAxes.Position = [7 213 489 235];

            % Create UIAxes_2
            app.UIAxes_2 = uiaxes(app.RightPanel);
            title(app.UIAxes_2, 'Title')
            xlabel(app.UIAxes_2, 'X')
            ylabel(app.UIAxes_2, 'Y')
            zlabel(app.UIAxes_2, 'Z')
            app.UIAxes_2.Position = [8 7 488 207];

            % Show the figure after all components are
               created
            app.UIFigure.Visible = 'on';
        end
    end
```

```
    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = EBBPrac

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```

5.2 MCU Code

```
#include <fmt/core.h>
#include <Arduino.h>
#include <iostream>
#include <sstream>
#include <vector>
#include <string>
#include <queue>

double P = 0;
double I = 0;
double D = 0;
double setpoint = 0;
double disturbance = 0;
double duty = 128; //50% duty cycle by default.
int control_outputs = 0; //By default only pwm1 is on.
bool start = 0; //Start off.
int message = 100; //Flags for certain messages.
int start_time = 0;

double temp1 = 0;
double temp2 = 0;
```

```
int N_samples = 200;
std::queue<double> samples1;
std::queue<double> samples2;

hw_timer_t *timer = NULL;
const int PWM_Out_1 = GPIO_NUM_32;
const int PWM_Out_2 = GPIO_NUM_25;
const int PWM_Meas_1 = GPIO_NUM_33;
const int PWM_Meas_2 = GPIO_NUM_26;

const int Temp_Meas1 = GPIO_NUM_27;
const int Temp_Meas2 = GPIO_NUM_35;

const double A = (3.3/4095.0);
const int PWM_freq = 10;
const int PWM_res = 8;

const int BaudRate = 1152000;
const int sampling_period = 5000; //In microseconds.

void setup() {
  //Start the USB-C serial/UART port.
  Serial.begin(BaudRate);

  //Setup I/O pins.
  pinMode(Temp_Meas1, INPUT);
  pinMode(Temp_Meas2, INPUT);
  pinMode(PWM_Meas_1, INPUT);
  pinMode(PWM_Meas_2, INPUT);

  ledcChangeFrequency(0, PWM_freq, PWM_res);
  ledcChangeFrequency(1, PWM_freq, PWM_res);
  ledcAttachPin (PWM_Out_1, 0);
  ledcAttachPin(PWM_Out_2, 1);

  timer = timerBegin(0, 80, true);

  samples1.push(0);
  samples2.push(0);
}

void loop() {

  //If receiving communication from computer, update values.
  if(Serial.available() > 0)
  {
    String received = Serial.readStringUntil('\n');
    received.trim();
```

```
std::istringstream stream(received.c_str());
std::string command;
std::string value_str;
double value;

std::getline(stream, command, ',');
std::getline(stream, value_str, ',');
value = std::stod(value_str);

if(command == "0")
{
  P = value;
}
else if(command == "1")
{
  I = value;
}
else if(command == "2")
{
  D = value;
}
else if(command == "3")
{
  setpoint = value;
}
else if(command == "4")
{
  disturbance = value;
}
else if(command == "5")
{
  duty = value;
  duty = (pow(2, PWM_res) - 1)*(duty/100.0); //Convert
     to percentage.
}
else if(command == "6")
{
  control_outputs = value;
  //Serial.print("Igot");
  //Serial.println(control_outputs);
}
else if(command == "7")
{
  start = value;

  //Check if start is true, then start timer.
  if(start)
```

```
    {
      start_time = millis();
    }
  }
}

//If start flag is true, start measuring inputs/start
   controlling aswell.
if(start)
{
  //Check when we started in order to see how long we were
      busy.
  timerRestart(timer);

  //Turn on the appropriate PWM signals.
  switch(control_outputs)
  {
    case 0:
      ledcWrite(0, duty);
      ledcWrite(1, 0);
      break;

    case 1:
      ledcWrite(1, duty);
      ledcWrite(0, 0);
      break;

    case 2:
      ledcWrite(0, duty);
      ledcWrite(1, duty);
      break;

      default:
      ledcWrite(0, 0);
      ledcWrite(1, 0);
  }

  //Interpret sensor data.
  double temp1_raw = (analogReadMilliVolts(Temp_Meas1)
     /1000.0 - 0.5)/0.01;
  double temp2_raw = (analogReadMilliVolts(Temp_Meas2)
     /1000.0 - 0.5)/0.01;

  if(samples1.size() > N_samples)
  {
    temp1 -= samples1.front();
    temp2 -= samples2.front();
```

```
      samples1.pop();
      samples2.pop();
    }

    temp1 += temp1_raw/N_samples;
    temp2 += temp2_raw/N_samples;

    samples1.push(temp1_raw/N_samples);
    samples2.push(temp2_raw/N_samples);

    //Send sensor input data as a comma seperated list.
    std::string sending = fmt::format("{0},␣{1},␣{2},␣{3},␣
       {4},␣{5}",
    temp1, temp2, analogRead(PWM_Meas_1)*A, analogRead(
       PWM_Meas_2)*A, millis() - start_time, message);

    Serial.println(String(sending.c_str()));
    Serial.flush();

    //Check how long everything took in microseconds.
    int elapsed = timerReadMicros(timer);

    if(elapsed <= sampling_period)
    {
      delayMicroseconds(sampling_period - elapsed);
      message = 100; //Reset message flag to show nothing.
    }
    else
    {
      delayMicroseconds(sampling_period);
      message = 0; //Set message flag for sampling too fast.
    }
  }
  //Otherwise deactivate PWM outputs.
  else
  {
    ledcWrite(0, 0);
    ledcWrite(1, 0);
  }
}
```

List of Tables

List of Figures